



RDS MULTI-AZ MIGRATION FROM AN ON-PREMISES MYSQL DATABASE

- Migration of an existing MySQL database to AWS
- Using Amazon RDS Multi-AZ for high availability
- Goal: Better reliability, backup, and disaster recovery





Project Team

The team is responsible for understanding the existing database challenges, identifying real-world use cases and analysing potential bottlenecks.



Amrutha hasini

Team Lead and Coordination

Overall project direction and alignment across workstreams.



Jijnesh

Problem Research and Use Case Identification

Investigating daily operational workloads and user touchpoints.



Rishyendra

Database Analysis and Bottleneck Identification

Evaluating compute, replication, and storage constraints.



Megha

Documentation and Industry Expert Discussions

Gathering external validation and structured industry feedback.

Problem Context – Current Situation



Existing Database Environment:

- The application currently depends on an on-premises MySQL database.
- Database infrastructure and maintenance are managed internally.
- Database availability depends on the reliability of existing infrastructure.
- Hardware failures and network disruptions may affect database access.

Core Concern:

The existing environment may experience availability, reliability and operational challenges during unexpected failures or maintenance activities.



Defining the Problem

How can applications depending on an on-premises MySQL database maintain reliable database access and minimise service interruptions during infrastructure failures, planned maintenance and unexpected disruptions?



Infrastructure

MySQL database runs on an on-premises server
Database depends on physical/virtual infrastructure

Risks

Hardware failure can cause downtime

Operations

Backups and disaster recovery require manual management
Scaling the database can be difficult



Use Cases Affected by the Problem

SCENARIO 01

Use Case 1: Normal Application Access

- Users and applications continuously access database records.
- Database interruptions may affect normal operations.

SCENARIO 02

Use Case 2: Peak Traffic

- Multiple users access the database simultaneously.
- Increased requests may cause performance degradation.

SCENARIO 03

Use Case 3: Planned Maintenance

- Database maintenance may require temporary service interruptions.
- Applications may experience restricted database access.

SCENARIO 04

Use Case 4: Unexpected Infrastructure Failure

- Hardware, storage or network failures may interrupt database availability.
- Users may lose access to application services.

SCENARIO 05

Use Case 5: Backup and Recovery

- Database failures may create concerns about data restoration.
- Untested backups may affect recovery reliability.



Identifying Existing Bottlenecks

USE CASE	POTENTIAL BOTTLENECKS
Normal Application Access	Single point of failure and limited availability
Peak Traffic	Resource limitations, slow queries and connection overload
Planned Maintenance	Service downtime and maintenance dependency
Unexpected Failure	Recovery delays and uncertain recovery time
Backup and Recovery	Backup gaps, untested restores and potential data loss

KEY OBSERVATION:

These bottlenecks can affect application performance, database availability and overall user experience.



Existing Constraints and Challenges

The following factors need investigation before defining the complete problem scope:



Database Compatibility

Current MySQL version and database size.



Downtime Tolerance

Acceptable application downtime during disruptions.



Recovery Requirements

Required Recovery Time Objective (RTO) and Recovery Point Objective (RPO).



Security Requirements

Network access, authentication and data protection.



Budget Constraints

Available project budget and infrastructure limitations.



Time Constraints

Available time for analysing and completing the project.



Comparison with Alternative Database Approaches

Database Approach	Limitation / Consideration
On-Premises MySQL	Requires internal infrastructure management and recovery planning.
Amazon RDS Single-AZ	Does not provide a standby instance in another Availability Zone.
MySQL Read Replica	Uses asynchronous replication and is primarily intended for read scaling.
Self-Managed MySQL on EC2	Requires the team to manage database infrastructure, replication and failover.
Amazon RDS Multi-AZ	Provides managed synchronous standby replication and automatic failover.



Mapping the Solution to Identified Problems

Existing Problem	How RDS Multi-AZ Helps
Database downtime	Automatic failover to the standby instance during supported failures.
Hardware failure	Standby instance is maintained in a separate Availability Zone.
Data reliability	Synchronous replication keeps committed updates on the standby.
Manual recovery effort	AWS manages standby maintenance and failover operations.
Planned maintenance	Certain maintenance operations can use the standby to reduce availability impact.
Infrastructure management	Managed database services reduce manual administrative work.



WHAT IS MULTI-AZ?

HIGH AVAILABILITY WITH MULTI-AZ

How It Works

- Multi-AZ creates a standby database in another Availability Zone
- Data is synchronously replicated from the primary database
- Standby automatically takes over during a failure
- Provides automatic failover
- Helps minimize database downtime

Replication Flow

Primary DB → Synchronous Replication → Standby DB





SERVICES UTILIZED

HIGH AVAILABILITY WITH MULTI-AZ

- Amazon RDS (MySQL) – Hosts the migrated relational database in a managed AWS environment.
- RDS Multi-AZ – Provides high availability using a primary and standby database across different Availability Zones.
- AWS DMS – Migrates and replicates data from the on-premises MySQL database to RDS.
- Amazon VPC – Provides an isolated and secure network environment for AWS resources.
- Amazon EC2 – Hosts the application/server that connects to the RDS database.
- Security Groups – Control inbound and outbound access to RDS and EC2.
- Amazon CloudWatch – Monitors database and application performance and AWS resource metrics.
- AWS IAM – Manages user access and permissions for AWS resources.





MIGRATION STEPS

IMPLEMENTATION PROCESS

01

Assess the existing MySQL database

02

Create an Amazon RDS MySQL instance

03

Configure networking and security

04

Use AWS Database Migration Service (DMS) for migration

05

Perform initial data load

06

Use continuous replication to reduce downtime

07

Validate migrated data

08

Switch application connection to RDS



SECURITY & BENEFITS

AFTER MIGRATION

Encryption

Encryption for data at rest and in transit

Access Control

IAM and security groups for access control

Failover

Multi-AZ failover

Monitoring

Monitoring and alerts

Management

Reduced infrastructure management

Backups

Automated backups

Availability

Improved availability and disaster recovery

